

ALGORITHMS AND PROBLEM SOLVING

(C LANGUAGE)

Internship Project Report | Alphido Tech

1. Project Objective

The objective of this project is to implement core algorithmic paradigms from scratch using the pure C programming language. This task specifically focuses on developing a Divide and Conquer sorting framework (QuickSort), a logarithmic search system (Binary Search), and an optimization framework based on the Greedy approach (Fractional Knapsack). Each algorithm is integrated with manual data input handling, dynamic dataset processing, and inline empirical execution runtime tracking utilizing system clock cycles.

2. Technologies & Modules Used

- **Language:** C (C99/C11 standard alignment)
- **Development Environment:** Visual Studio Code (VS Code)
- **Compiler:** GCC (GNU Compiler Collection)
- **Core Libraries Used:**
 - `<stdio.h>` for Standard Input/Output console operations.
 - `<stdlib.h>` for Dynamic memory allocations via `malloc` and `free`, and random number generation via `rand` and `srand`.
 - `<time.h>` for capturing low-level processor clock ticks to calculate execution runtimes objectively.

3. Project Description

This project demonstrates the engineering of optimized problem-solving logic, comparing theoretical asymptotic bounds against empirical execution data. All routines execute without external framework dependencies, relying completely on close-to-the-metal C structural logic, pointers, and manual heap allocations.

Core Algorithmic Frameworks:

- **QuickSort Framework (01_Quick_sort.c):**
 - **Strategy:** Divide and Conquer partitioning.

- **Logic:** Selects the boundary element as a pivot, shifting elements smaller than the pivot to the left and larger elements to the right. It then processes the remaining sub-arrays recursively.
- **Profiling Component:** Generates randomized large datasets and benchmarks sorting time intervals dynamically using system clock cycles.

```

C 01_Quick_sort.c
C 01_Quick_sort.c > swap(int*,int*)
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h> // For measuring execution runtime
4
5 // Function to swap two elements
6 void swap(int* a, int* b) {
7     int temp = *a;
8     *a = *b;
9     *b = temp;
10 }
11 // Partition function: places the pivot element at its correct position
12 int partition(int arr[], int low, int high) {
13     int pivot = arr[high]; // Choosing the last element as pivot
14     int i = (low - 1); // Index of smaller element
15
16     for (int j = low; j <= high - 1; j++) {
17         if (arr[j] < pivot) {
18             i++;
19             swap(&arr[i], &arr[j]);
20         }
21     }
22     swap(&arr[i + 1], &arr[high]);
23     return (i + 1);
24 }
25
26 // The QuickSort Algorithm
27 void quickSort(int arr[], int low, int high) {
28     if (low < high) {
29         int pi = partition(arr, low, high); // Partitioning index
30
31         quickSort(arr, low, pi - 1); // Sort elements before partition
32         quickSort(arr, pi + 1, high); // Sort elements after partition
33     }
34 }
35
36 int main() {
37     // ...
38 }
39
40 #endif

```

- **Binary Search Framework (02_Binary_search.c):**
 - **Strategy:** Iterative Logarithmic Lookup.
 - **Logic:** Operates strictly on sorted array inputs. To guarantee correct runtime execution for any arbitrary user inputs, the pipeline runs a foundational sorting phase before executing the search logic, repeatedly dividing the target lookup boundaries in half.

```

C 02_Binary_search.c
C 02_Binary_search.c > main()
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Standard Bubble Sort function to prepare user inputs for Binary Search
5 void sortArray(int arr[], int n) {
6     for (int i = 0; i < n - 1; i++) {
7         for (int j = 0; j < n - i - 1; j++) {
8             if (arr[j] > arr[j + 1]) {
9                 int temp = arr[j];
10                arr[j] = arr[j + 1];
11                arr[j + 1] = temp;
12            }
13        }
14    }
15 }
16
17 // Standard binary search logic
18 int binarySearch(int arr[], int n, int item) {
19     int left = 0;
20     int right = n - 1;
21
22     while (left <= right) {
23         int mid = left + (right - left) / 2;
24
25         if (arr[mid] == item) {
26             return mid; // Found it
27         }
28
29         if (arr[mid] < item) {
30             left = mid + 1; // Look in right half
31         }
32     }
33
34     return -1; // Not found
35 }
36
37 int main() {
38     // ...
39 }
40
41 #endif

```

- **Fractional Knapsack Framework (03_knapsack.c):**

- **Strategy:** Optimization via Greedy Choice.
- **Logic:** Leverages custom structure definitions (struct Item) to store values and weights. It sorts items in descending order based on their value-to-weight efficiency ratios and adds elements into the container up to maximum capacity, seamlessly computing fractional splits when space constraints are reached.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Item {
5      int value;
6      int weight;
7  };
8
9  // Sorting items based on value per kg ratio (descending)
10 void sortItemsByRatio(struct Item items[], int n) {
11     for (int i = 0; i < n - 1; i++) {
12         for (int j = 0; j < n - i - 1; j++) {
13             double ratio1 = (double)items[j].value / items[j].weight;
14             double ratio2 = (double)items[j + 1].value / items[j + 1].weight;
15
16             // Sort in descending order
17             if (ratio1 < ratio2) {
18                 struct Item temp = items[j];
19                 items[j] = items[j + 1];
20                 items[j + 1] = temp;
21             }
22         }
23     }
24 }
25
26 // Greedy Fractional Knapsack Algorithm implementation
27 double fractionalKnapsack(int capacity, struct Item items[], int n) {
28     sortItemsByRatio(items, n);
29
30     double totalValue = 0.0;
31     int currentWeight = 0;
32
33     // ... (rest of the implementation)

```

4. Code Execution Verification (Sample Input/Output)

A. QuickSort Execution Template (01_Quick_sort.c)

```

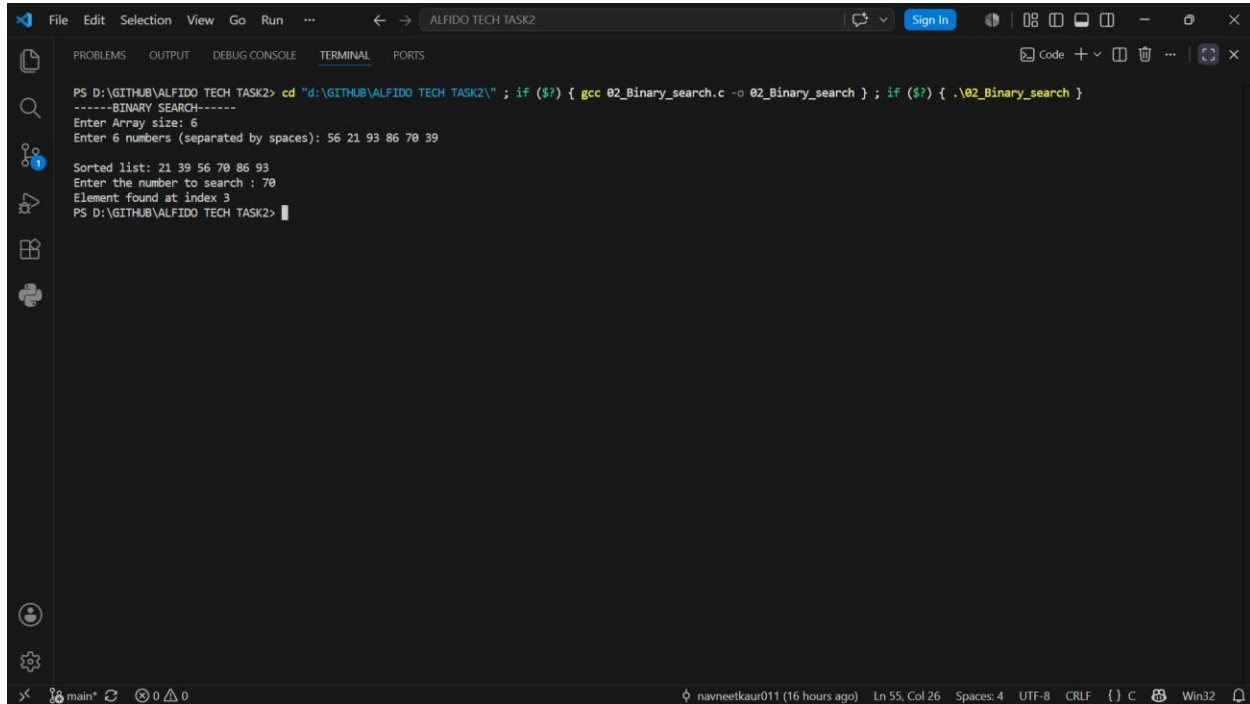
PS D:\GITHUB\ALFIDO TECH TASK2> cd "d:\GITHUB\ALFIDO TECH TASK2\" ; if ($?) { gcc 01_Quick_sort.c -o 01_Quick_sort ; if ($?) { .\01_Quick_sort }
-----QUICKSORT RUNTIME-----
Enter the number of elements to sort: 634521

Sorting Started, Please wait...
Sorting done!
Time Taken: 109.00 milliseconds.

PS D:\GITHUB\ALFIDO TECH TASK2>

```

B. Binary Search Execution Template (02_Binary_search.c)



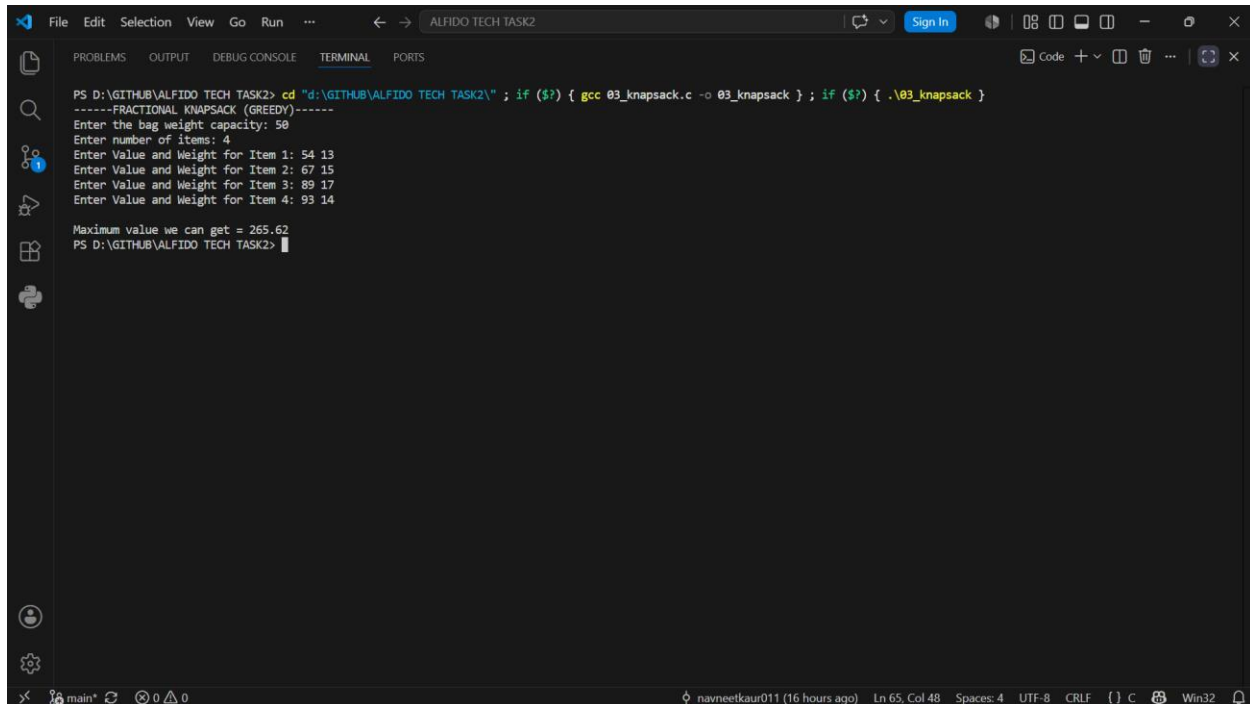
The screenshot shows a VS Code terminal window with the following content:

```
PS D:\GITHUB\ALFIDO TECH TASK2> cd "d:\GITHUB\ALFIDO TECH TASK2\" ; if ($?) { gcc 02_Binary_search.c -o 02_Binary_search } ; if ($?) { .\02_Binary_search }
-----BINARY SEARCH-----
Enter Array size: 6
Enter 6 numbers (separated by spaces): 56 21 93 86 70 39

Sorted list: 21 39 56 70 86 93
Enter the number to search : 70
Element found at index 3
PS D:\GITHUB\ALFIDO TECH TASK2>
```

The terminal window has a dark theme and includes a sidebar with icons for Explorer, Search, Run and Debug, and Extensions. The bottom status bar shows the file path, line and column numbers (Ln 55, Col 26), and the encoding (UTF-8).

C. Fractional Knapsack Execution Template (03_knapsack.c)



The screenshot shows a VS Code terminal window with the following content:

```
PS D:\GITHUB\ALFIDO TECH TASK2> cd "d:\GITHUB\ALFIDO TECH TASK2\" ; if ($?) { gcc 03_knapsack.c -o 03_knapsack } ; if ($?) { .\03_knapsack }
-----FRACTIONAL KNAPSACK (GREEDY)-----
Enter the bag weight capacity: 50
Enter number of items: 4
Enter Value and Weight for Item 1: 54 13
Enter Value and Weight for Item 2: 67 15
Enter Value and Weight for Item 3: 89 17
Enter Value and Weight for Item 4: 93 14

Maximum value we can get = 265.62
PS D:\GITHUB\ALFIDO TECH TASK2>
```

The terminal window has a dark theme and includes a sidebar with icons for Explorer, Search, Run and Debug, and Extensions. The bottom status bar shows the file path, line and column numbers (Ln 65, Col 48), and the encoding (UTF-8).

5. Key Learning Outcomes

- **Empirical Performance Profiling:** Learned how to use system clock cycles via `<time.h>` to capture actual execution runtimes, bridging the gap between theoretical Big-O complexity bounds and physical code execution speeds.
- **Divide-and-Conquer Control Flows:** Mastered structural recursive design patterns in pure C, optimizing dynamic stack trace flows and partitioning boundaries without relying on abstract C++ collections or containers.
- **Greedy Optimization Implementation:** Successfully applied structural optimization heuristics by designing custom multi-property definitions (struct), computing density ratios on the fly, and handling fractional value division smoothly using floating-point math logic.
- **Pre-requisite Validation Pipelines:** Gained experience building robust implementation wrappers, such as ensuring data is fully sorted via a reliable preprocessing sorting step before calling binary search routines.

6. Conclusion

This task provided comprehensive, close-to-the-metal programming experience in classic sorting, searching, and greedy optimization algorithms. By engineering these core paradigms entirely within the structural rules of the C language rather than utilizing pre-packaged library wrappers, it reinforced a deep understanding of memory addressing layout constraints, pointer swapping logic, and algorithmic efficiency. These skills are essential for high-performance systems engineering and advanced software design.

Prepared By:

- **Name:** Navneet Kaur
- **Candidate ID:** BS/REG/123254
- **Role:** C/C++ Intern
- **Company:** Alphido Tech
- **Degree:** Bachelor of Computer Applications